Arguments in Python Functions

Arguments are values passed to a function when it is called.

```
def greet(name):
    print("Hello", name)

greet("Neeraj")
```

Types of Arguments

1. Positional Arguments

- Passed in the correct order

```
def add(a, b):
    print(a + b)

add(5, 3)   # a=5, b=3
```

2. Keyword Arguments

- Passed using parameter names

```
def student(name, age):
    print(name, age)

student(age=21, name="Rahul")
```

3. Default Arguments

- Predefined values if no argument is passed

```
def greet(name="Guest"):
    print("Hello", name)

greet()          # Hello Guest
greet("Neeraj")  # Hello Neeraj
```

4. Variable-Length Arguments (*args)

- Accept multiple values (tuple)

```
def total(*numbers):
    print(sum(numbers))

total(1, 2, 3, 4)
```

5. Keyword Variable Arguments (**kwargs)

- Accept multiple key-value pairs (dictionary)

```
def info(**data):
    print(data)

info(name="Neeraj", age=23)
```

Recursion in Python

A function calling itself is called recursion.

Example: Factorial

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))  # 120
```

Important Points

- Must have a **base case**
- Otherwise → infinite loop

Example: Fibonacci

```
def fib(n):
    if n <= 1:
        return n
    return fib(n-1) + fib(n-2)

print(fib(6))  # 8
```

Creating Custom Modules

A file containing Python code (functions, variables)

Step 1: Create a File (mymodule.py)

```
def add(a, b):
    return a + b

def greet(name):
    return "Hello " + name
```

Step 2: Use the Module

```
import mymodule

print(mymodule.add(2, 3))
print(mymodule.greet("Neeraj"))
```

Import Specific Function

```
from mymodule import add

print(add(5, 6))
```

Virtual Environment & Package Management (pip)

- Separate environment for each project
- Avoids package conflicts

Create Virtual Environment

```
python -m venv myenv
```

Activate Environment

Windows:

```
myenv\Scripts\activate
```

Mac/Linux:

```
source myenv/bin/activate
```

Install Packages using pip

```
pip install numpy
pip install flask
```

Check Installed Packages

```
pip list
```

Save Dependencies

```
pip freeze > requirements.txt
```

Install from File

```
pip install -r requirements.txt
```

Deactivate Environment

```
deactivate
```

Errors in Python (Syntax vs Runtime)

Syntax Errors

- Occur when Python rules are not followed
- Program **does not run**

```
def func()
    print("Hello")
```

Missing : → SyntaxError

Runtime Errors (Exceptions)

- Occur while the program is running
- Program starts but crashes

```
a = 10
b = 0
print(a / b)
```

Output: `ZeroDivisionError`

2. try & except

To handle errors and prevent program crash

```
try:
    a = int(input("Enter number: "))
    print(10 / a)
except:
    print("Error occurred!")
```

Specific Exception Handling (Recommended)

```
try:
    a = int(input("Enter number: "))
    print(10 / a)
except ZeroDivisionError:
    print("Cannot divide by zero")
except ValueError:
    print("Invalid input")
```

Multiple except Blocks

- Used to handle different errors separately

```
try:
    x = int("abc")
    y = 10 / 0
except ValueError:
    print("Value Error occurred")
except ZeroDivisionError:
    print("Division by zero")
```

else & finally

else Block

- Runs if **no error occurs**

finally Block

- Runs **always** (error or nor error)

```
try:
    a = 10 / 2
except ZeroDivisionError:
    print("Error")
else:
    print("Success:", a)
finally:
    print("This always runs")
```

Raising Exceptions (raise)

To manually generate an error

```
age = int(input("Enter age: "))

if age < 18:
    raise Exception("You are not eligible")
else:
    print("Eligible")
```

Raise Specific Exception

```
num = -5

if num < 0:
    raise ValueError("Number cannot be negative")
```

Custom Exceptions

User-defined error class

Example

```
class MyError(Exception):
    pass

try:
    raise MyError("This is a custom error")
except MyError as e:
    print(e)
```

Real-Life Example

```
class AgeError(Exception):
    pass

age = int(input("Enter age: "))

if age < 18:
    raise AgeError("Underage not allowed")
```

```
else:
    print("Access granted")
```

Polymorphism in Python

- Same function name → different behavior
- "One name, many forms"

Duck Typing 🦆

- Python does not care about object type
- It only cares about methods

```
class Dog:
    def speak(self):
        print("Bark")

class Cat:
    def speak(self):
        print("Meow")

def make_sound(animal):
    animal.speak()

d = Dog()
c = Cat()

make_sound(d)
make_sound(c)
```

If it behaves like a duck, it is a duck!

Operator Overloading

- Same operator behaves differently

```
print(5 + 3)        # 8
print("Hello " + "World")  # Hello World
```

Custom Operator Overloading

```
class Number:
    def __init__(self, value):
        self.value = value

    def __add__(self, other):
        return self.value + other.value

n1 = Number(10)
n2 = Number(20)

print(n1 + n2)  # 30
```

Abstract Classes & Interfaces (abc module)

What is Abstract Class?

- Cannot create object directly
- Used as blueprint

Example using abc module

```
from abc import ABC, abstractmethod

class Shape(ABC):

    @abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def area(self):
        return 10 * 5

r = Rectangle()
print(r.area())
```

Key Points

- `ABC` → Abstract Base Class
- `@abstractmethod` → must be implemented
- Child class must define all abstract methods

Iterators & Generators

Iterator

- Object that can be looped
- Uses `iter()` and `next()`

```
nums = [1, 2, 3]

it = iter(nums)

print(next(it))  # 1
print(next(it))  # 2
print(next(it))  # 3
```

Generator

- Function that returns values one by one
- Uses `yield`

```
def count(n):
    for i in range(n):
        yield i

gen = count(5)
```

```
for num in gen:
    print(num)
```

Difference

Iterator	Generator
Uses iter()	Uses yield
Manual control	Automatic
More memory	Memory efficient

Comprehensions in Python

1. List Comprehension

```
squares = [x*x for x in range(5)]
print(squares)
```

2. Dictionary Comprehension

```
d = {x: x*x for x in range(5)}
print(d)
```

3. Set Comprehension

```
s = {x*x for x in range(5)}
print(s)
```

4. Generator Comprehension

```
g = (x*x for x in range(5))

for i in g:
    print(i)
```

With Condition

```
even = [x for x in range(10) if x % 2 == 0]
print(even)
```